Práctica ORM-SQLAlchemy-Audios

Entrega: guardar información de audios en SQLite con SQLAlchemy (CRUD) y guardar los datos reales de **0802.mp3** con pydub.

Archivos de la práctica

• **practica.py** — crear tabla, insertar, leer, actualizar y borrar.
• **guardar_mp3.py** — leer **0802.mp3** y guardar frecuencia, bits y duración.

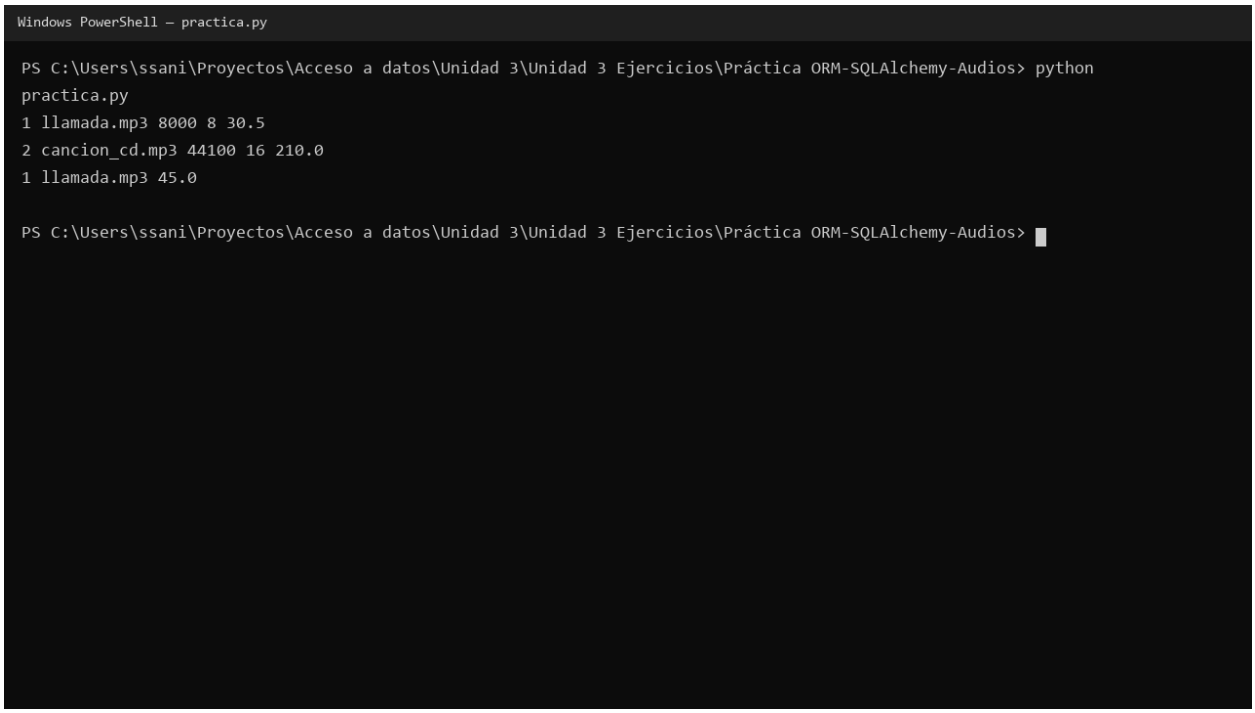
1. Código de practica.py

```
from sqlalchemy import create_engine, Column, Integer, String, Float
from sqlalchemy.orm import declarative_base, Session

engine = create_engine("sqlite:///audios.db")
Base = declarative_base()

class Audio(Base):
    __tablename__ = "audios"
    id = Column(Integer, primary_key=True)
    nombre = Column(String)
    frecuencia = Column(Integer)
    bits = Column(Integer)
    duracion = Column(Float)

Base.metadata.drop_all(engine)
Base.metadata.create_all(engine)

with Session(engine) as sesion:
    a1 = Audio(nombre="llamada.mp3", frecuencia=8000, bits=8, duracion=30.5)
    a2 = Audio(nombre="cancion_cd.mp3", frecuencia=44100, bits=16, duracion=210.0)
    sesion.add(a1)
    sesion.add(a2)
    sesion.commit()

with Session(engine) as sesion:
    audios = sesion.query(Audio).all()
    for audio in audios:
        print(audio.id, audio.nombre, audio.frecuencia, audio.bits, audio.duracion)

with Session(engine) as sesion:
    audio = sesion.query(Audio).filter_by(nombre="llamada.mp3").first()
    audio.duracion = 45.0
    sesion.commit()

with Session(engine) as sesion:
    audio = sesion.query(Audio).filter_by(nombre="cancion_cd.mp3").first()
    sesion.delete(audio)
    sesion.commit()

with Session(engine) as sesion:
    for audio in sesion.query(Audio).all():
        print(audio.id, audio.nombre, audio.duracion)
```

2. Ejecución de practica.py

Salida esperada del CRUD (crear, leer, actualizar y borrar):

```
1 llamada.mp3 8000 8 30.5
2 cancion_cd.mp3 44100 16 210.0
1 llamada.mp3 45.0
```


Windows PowerShell — practica.py
PS C:\Users\ssani\Proyectos\Acceso a datos\Unidad 3\Unidad 3 Ejercicios\Práctica ORM-SQLAlchemy-Audios> python practica.py
1 llamada.mp3 8000 8 30.5
2 cancion_cd.mp3 44100 16 210.0
1 llamada.mp3 45.0
PS C:\Users\ssani\Proyectos\Acceso a datos\Unidad 3\Unidad 3 Ejercicios\Práctica ORM-SQLAlchemy-Audios>

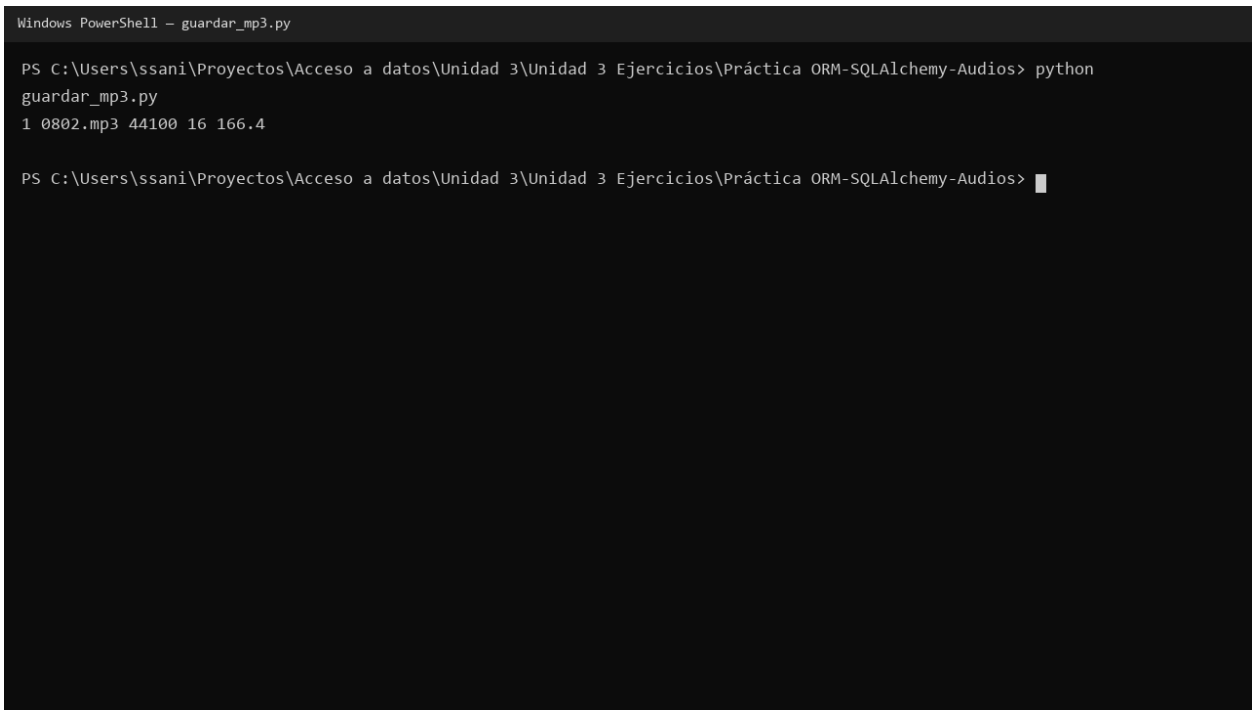
3. Código de guardar_mp3.py

```
from pydub import AudioSegment
from sqlalchemy import create_engine, Column, Integer, String, Float
from sqlalchemy.orm import declarative_base, Session

engine = create_engine("sqlite:///audios.db")
Base = declarative_base()

class Audio(Base):
    __tablename__ = "audios"
    id = Column(Integer, primary_key=True)
    nombre = Column(String)
    frecuencia = Column(Integer)
    bits = Column(Integer)
    duracion = Column(Float)

Base.metadata.drop_all(engine)
Base.metadata.create_all(engine)

sonido = AudioSegment.from_mp3("0802.mp3")
with Session(engine) as sesion:
    nuevo = Audio(
        nombre="0802.mp3",
        frecuencia=sonido.frame_rate,
        bits=sonido.sample_width * 8,
        duracion=len(sonido) / 1000,
    )
    sesion.add(nuevo)
    sesion.commit()

with Session(engine) as sesion:
    for audio in sesion.query(Audio).all():
        print(audio.id, audio.nombre, audio.frecuencia, audio.bits, audio.duracion)
```

4. Ejecución de guardar_mp3.py

Datos reales del archivo **0802.mp3** guardados en la base de datos:

```
1 0802.mp3 44100 16 166.4
```



5. Conclusión

Con SQLAlchemy se ha realizado el CRUD completo sobre la tabla **audios** y se han guardado los metadatos del MP3 del ejercicio (44100 Hz, 16 bits, 166.4 s) sin escribir SQL a mano.